How can we perform generic conditional synchronization with semaphores in a similar (and simple) way as condition variables?

To achieve this behavior we should separate two different scenarios:

(1) There is no distinction among the active waiting entities, meaning that any one of them can be the target of the signaling process (e.g., inserting or removing an element from a queue).
In this simpler scenario, since semaphores are essentially (special!) shared counting variables, we can achieve the desired equivalence by counting the number of active waiting entities. This solution works because the conditional synchronization semaphore (cvar_id) is used only when *cvar_waiters is positive, and *cvar_waiters is positive only when the condition is not met.
If a separate conditional synchronization semaphore is used for each condition (as should also be done with condition variables), no broadcast is required; a single signal is sufficient and preferred.

MUTEX & CONDITION VARIABLES

```
#include <pthread.h>
#include <hdp>

// INITIALIZATION:

// mutex:
pthread_mutex_t mtx;
NZ(pthread_mutex_init(&mtx, NULL));

// condition variable:
pthread_cond_t cvar;
NZ(pthread_cond_init(&cvar, NULL));



// WAIT:

NZ(pthread_mutex_lock(&mtx));
while (!condition)
{

   NZ(pthread_cond_wait(&cvar,&mtx));

}
...
NZ(pthread_mutex_unlock(&mtx));



// SIGNAL:
NZ(pthread_mutex_lock(&mtx));
...

NZ(pthread_cond_signal(&cvar));

NZ(pthread_mutex_unlock(&mtx));



// BROADCAST:
NZ(pthread_mutex_lock(&mtx));
...

NZ(pthread_cond_broadcast(&cvar));

NZ(pthread_mutex_unlock(&mtx));
```

SHARED MEMORY & SEMAPHORES

```
#include <sys/shm.h>
#include <hdp.h>
#include <semext.h>

// Binary semaphore with initial value 1:
int mtx_id = MO(semget(semkey, 1, 0600 | IPC_CREAT | IPC_EXCL));
semup(mtx_id, 0);

// Integer semaphore with initial value 0:
int cvar_id = MO(semget(key, 1, 0600 | IPC_CREAT | IPC_EXCL));

// Shared integer variable (in shared memory!):
int shmid = MO(shmget(shmkey, sizeof(int), 0600 | IPC_CREAT | IPC_EXCL));
int* cvar_waiters = (int*)MOP(shmat(shmid, NULL, 0)); // before fork(s)
*cvar_waiters = 0;
...
MO(shmdt(cvar_waiters)); // at the end, after all wait[pid]



// wait (assumes cvar_waiters attached):
semdown(mtx_id, 0); // lock
while (!condition)
{
   (*cvar_waiters)++;
   semup(mtx_id, 0); // unlock
   semdown(cvar_id, 0); // wait
   semdown(mtx_id, 0); // lock
}
...
semup(mtx_id, 0); // unlock



// signal (assumes cvar_waiters attached):
semdown(mtx_id, 0); // lock
...
if((*cvar_waiters) > 0)
{
   semup(cvar_id, 0); // signal one
   (*cvar_waiters)--;
}
semup(mtx_id, 0); // unlock



// broadcast (assumes cvar_waiters attached):
semdown(mtx_id, 0); // lock
...
while((*cvar_waiters) > 0)
{
   semup(cvar_id, 0); // signal one
   (*cvar_waiters)--;
}
semup(mtx_id, 0); // unlock
```

(2) When the waiting active entities might not all be applicable to a signal (eg. withdraw money from a shared account), then the previous implementation might fail. The problem arises because the same active entity might respond twice (or more times) to the same broadcast (no control there, because we have only a cvar waiting counter); thus preventing one waiting active entity to respond to a broadcast.
In this case, a different much more complex alternative is required (not approached here because it will not be required for this course).